UML Component Diagram — Hostel Maintenance & Issue Ticketing System

Lab 3: Component Modelling & Architectural Pattern Selection | Architecture: Layered | SRN: PES1UG24CS375 | SE Section G | Faculty: Dr. Sapna V M

Presentation Layer

«component»

Ticket Portal UI Component

Web/mobile portal used by Resident, Warden and Staff

Ticket form (room tag, photos, severity) · dashboard · sign-off

«interface» ITicketService

createTicket() · assignTicket() · getStatus() · confirmClosure()

REST / HTTPS (JSON) ↓ ticket form, actions ↑ ticket ID, status

«use»

Business Layer

«interface» IAccessControl

«component»

Access Control Component

- Authenticate users (session / token)
- Role-based access control (NFR-002)
- Guard photos & contact details

authenticate()
authorize(role, res)

in-process API call

«component»

Ticket Manager Component

- Validate & create tickets with unique ID (FR-001)
- Assign by category & staff availability (FR-002)
- Track real-time status for dashboard (FR-003)
- Enforce resident sign-off before “Closed” (FR-004)
- Record staff remarks, time spent, parts used (FR-005)
- Monitor 24-hour SLA and trigger escalation (NFR-001)

«interface» INotification

«component»

Notification Service Component

- SMS / Email delivery
- Assignment notice ≤ 1 min (FR-002)
- SLA escalation alerts (NFR-001)

sendAlert()
escalateSLA()

SMTP / SMS gateway API

«use»

«interface» IDataAccess

save() / query() tickets, photos, users, remarks

SQL via JDBC/ORM ↓ INSERT/UPDATE/SELECT ↑ result sets

Data Layer

«component»

Database Component

- Tickets, status history, remarks, time & parts used
- Photo attachments, users, room / block registry
- Staff categories & availability

Layering rule:

each layer calls only the layer directly below it, always through a declared interface. The UI never accesses the Database directly.

Legend

- Provided (ball) ⟷ Required (socket) interface = assembly connector
- «use» dependency: direction of request / data flow (consumer → provider)
- Port (interaction point on boundary)

Architectural Justification — Lab 3

Hostel Maintenance & Issue Ticketing System | SRN: PES1UG24CS375 | SE Section G | Faculty: Dr. Sapna V M

1. Architecture Selection

We chose **Layered Architecture** for the **Hostel Maintenance & Issue Ticketing System**. The system is split into a Presentation Layer (Ticket Portal UI), a Business Layer (Ticket Manager, Access Control, Notification Service) and a Data Layer (Database), connected through four interfaces: ITicketService, IAccessControl, INotification and IDataAccess.

Style	Assessment for this system	Verdict
Layered	Clear separation of UI, ticket rules and storage; simple to build, test and maintain; single deployable unit.	Selected
Microservices	Independent scaling and fault isolation, but adds network calls, service discovery and distributed data consistency — overkill for one hostel with three user roles.	Rejected
Client-Server	Central control and easy consistency, but it only describes the client/server split, not how ticket logic, RBAC and alerts are structured; the single server is a single point of failure.	Rejected

2. Two Reasons for This Choice

Reason 1 — The ticket workflow is a natural top-down pipeline. Every use case flows from a user action (log ticket, view status, sign-off) through business rules (validation, category-based assignment, SLA checks) to persistence. The requirements map cleanly onto the layers: FR-001/003/004 screens in the Presentation Layer; FR-002, FR-005 and NFR-001 in the Ticket Manager; storage in the Database Component.

Reason 2 — It matches the scale, team and maintainability needs. This is a single-campus system with modest ticket volume and a small development team. A layered, single-deployment design is far easier to build, test and operate than distributed services, while the interface contracts still keep components replaceable — for example, the SMS/Email provider behind INotification can be swapped without changing the Ticket Manager.

3. Security Advantage

Centralised, enforceable access control (NFR-002). Every request must pass through the Business Layer, where the Ticket Manager calls IAccessControl before touching any data. Role-based checks are therefore applied in one place, so only the assigned Warden and Staff can retrieve photo attachments and resident contact details; any other role receives a denied response. Because the UI cannot reach the Database directly, and the Database is exposed only through IDataAccess, it can sit on a private network segment, and the single enforcement point is easy to audit and test.

4. Performance Benefit

Low-latency internal calls meet the time targets. Ticket Manager, Access Control and Notification run in the same deployment and communicate through direct in-process interface calls, avoiding network hops between components. The only external calls are database queries and the SMS/Email gateway. This comfortably meets the 1-minute assignment notification (FR-002) and 2-minute dashboard refresh (FR-003) targets. Notification dispatch can run asynchronously so ticket submission is never blocked, and a scheduled SLA check in the Business Layer supports the 24-hour escalation rule (NFR-001). The extra layer-traversal overhead of this style is negligible at hostel scale.